CE 791 HPC - Mini Project
2D Groundwater Flow with FD

Andrew Navratil

The 2D groundwater flow equation is coded in Fortran using finite differences. The forward Euler method is utilized for time stepping, using the provided finite difference approximation formula. The delta x and delta y values are kept separate and allowed to differ. The time step value utilized is 0.001 days after experimenting with various values (no stability analysis was performed on the equation to get a CFL number). The transmissivity function coefficients are calculated using Matlab code with the conditions provided, giving values of a = -0.001, b = -0.002, c = 0.000004, and d = 2.0. The boundary conditions are implemented as given, with homogeneous Neumann on the top and bottom, and nonhomogeneous Dirichlet on the left and right.

Parallelization is performed in MPI using 1D domain decomposition. The 2D physical domain is partitioned into 1D blocks of columns, where the number of interior column nodes is specified to be divisible by all numbers of processors utilized, resulting in nx = 16*64, splitting the 1000m length. A derived datatype for columns is created to enable passing ghost columns to left and right neighbor processors when calling MPI_SENDRECV. The multidimensional array storing the solution is purposely organized so that columns of the physical x-y domain are also columns in the solution i-j multidimensional array, and the derived type utilizes MPI_TYPE_CONTIGUOUS. A virtual topology is created using MPI_CART_CREATE.

Performance plots are generated with Matlab, and the program is executed on henry2. The execution time and mflops calculations are executed in Fortran for each run combo of num procs (1,2,4,8,16) and overall time (10, 100, 1000 days). The values are printed to gw.out and manually copied into Matlab code for plotting. The number of operations per grid point per iteration is estimated at 33 for the FDA.

Solution plots are also generated with Matlab for each of the three overall times. The Fortran code writes out the solution to a txt file which is then imported into Matlab and plotted using contours. The execution using 16 processors was utilized for producing the output data, with the additional goal of confirming the parallelization code.

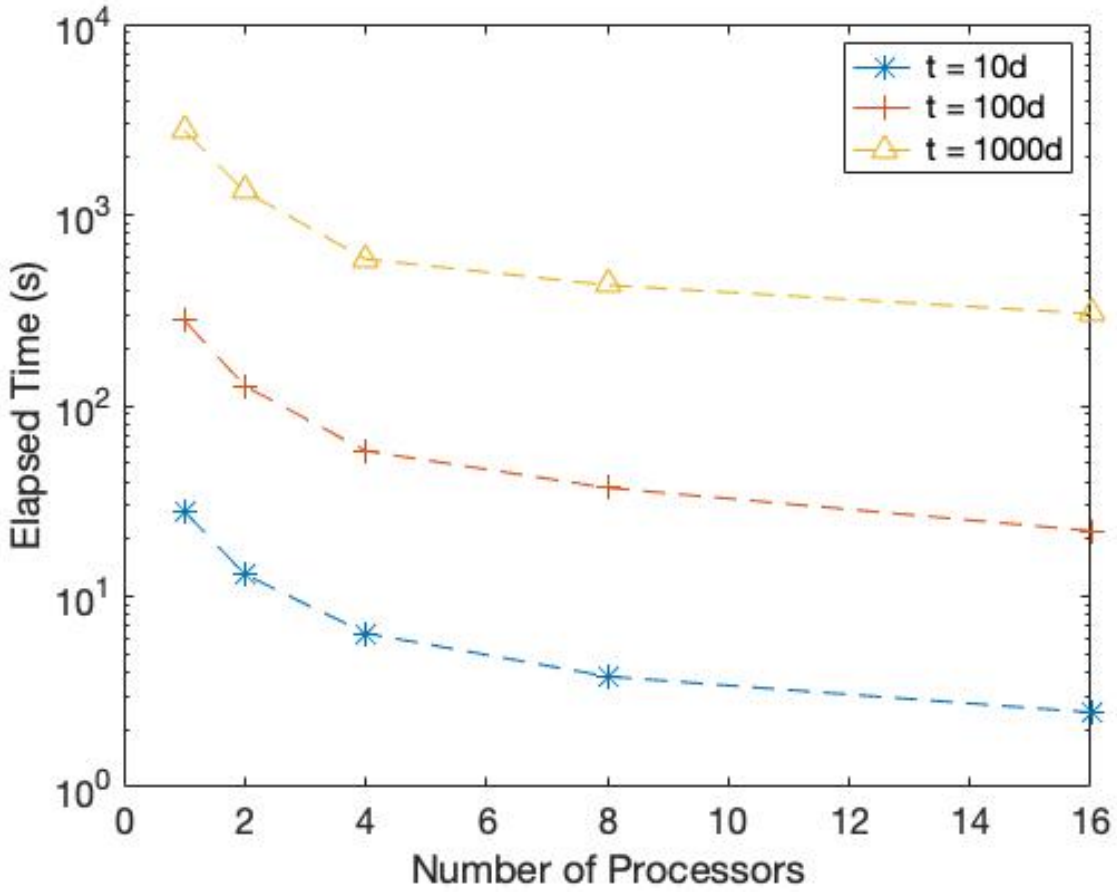

Fig. 1 Execution Times

The plot of execution times is done with a semi-log plot, with a log y-axis, in order to combine all three number of days together. The expected trend is observed for a properly parallelized code, where execution time decreases as number of processors increases, and overall execution time increases as the number of days increases, using the same time step to ensure stability since the grid is the same.

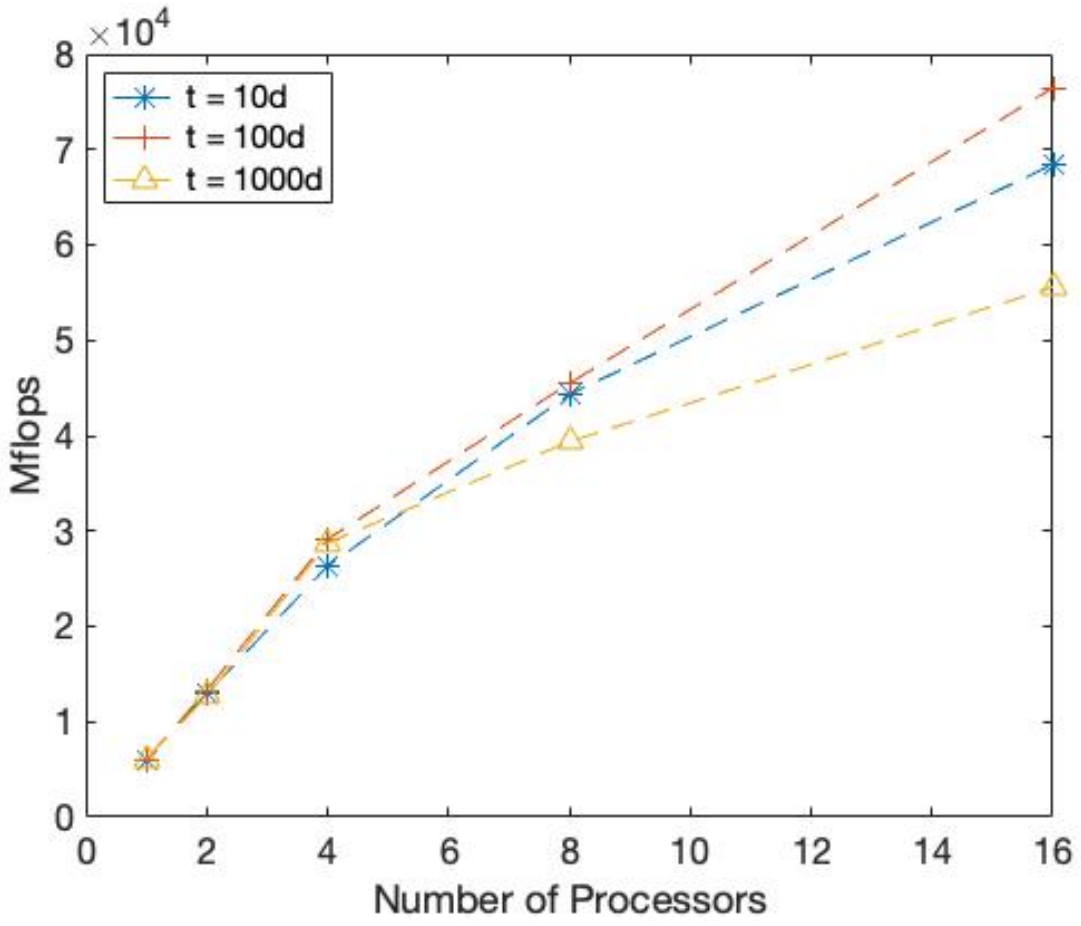

Fig. 2 Mflops

The plot of Mflops also shows the expected trend for a properly parallelized code. The Mflops increases as the number of processors is increased, and this increase is at almost the same rate regardless of the amount of time it is run (both total number of ops and total execution times increase giving similar ratios for Mflops). The performace does start to diverge around 8 processors, with an increase for 100 days and decrease for 1000 days. The maximum performance seen at 16 processors is around 75 Gflops.

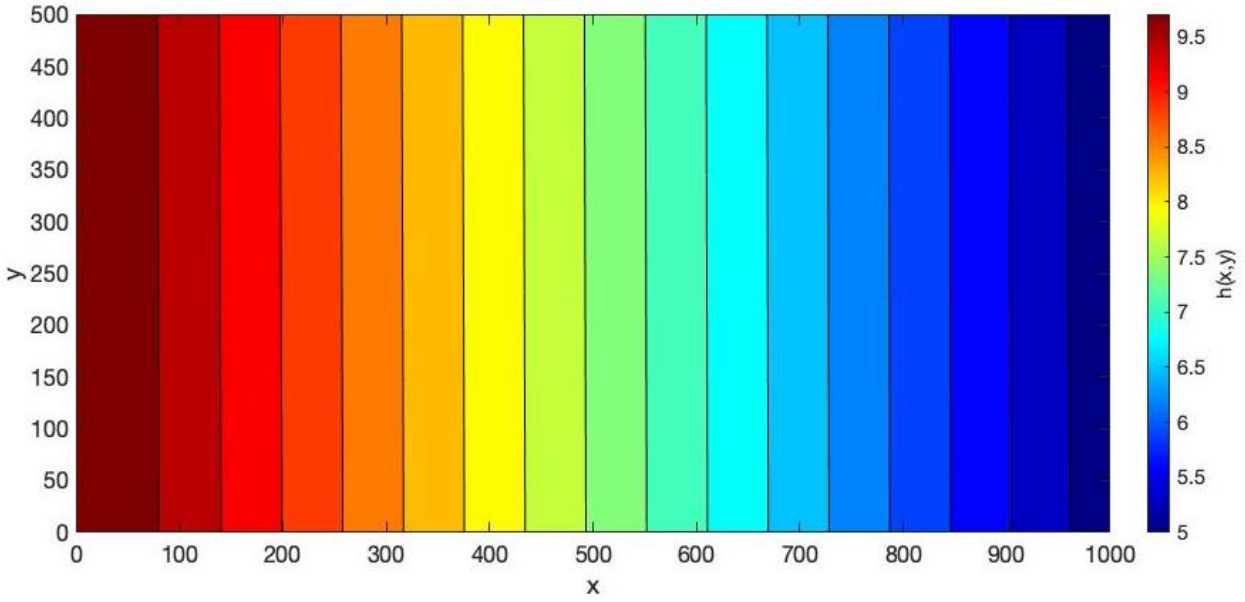

Fig. 3 Solution at 10 days

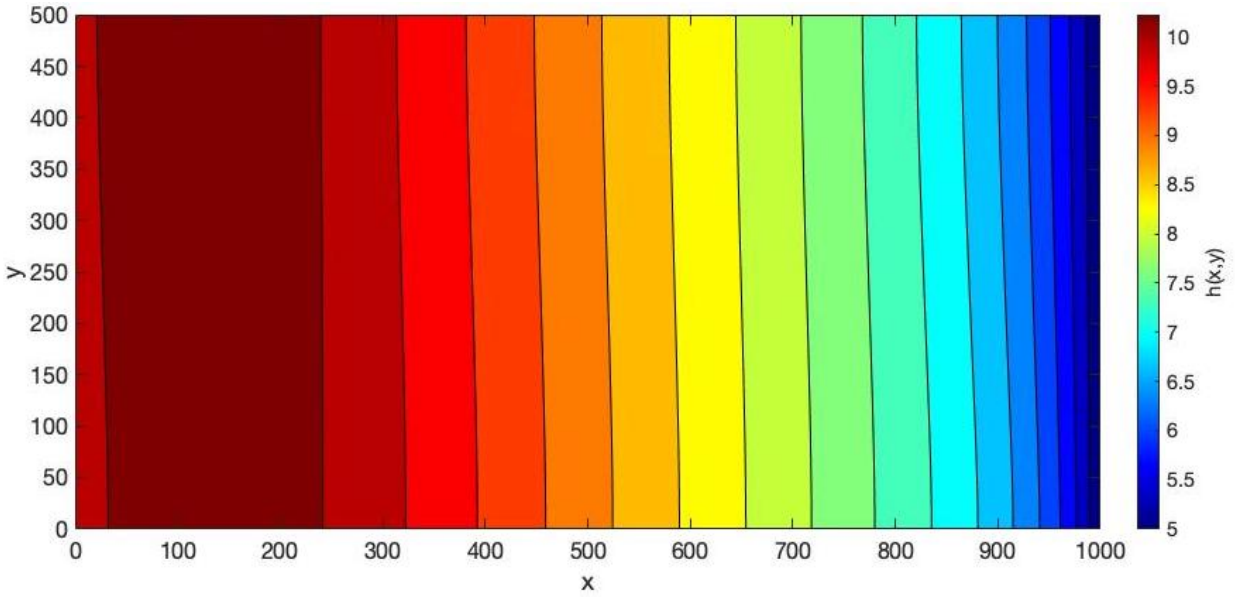

Fig. 4 Solution at 100 days

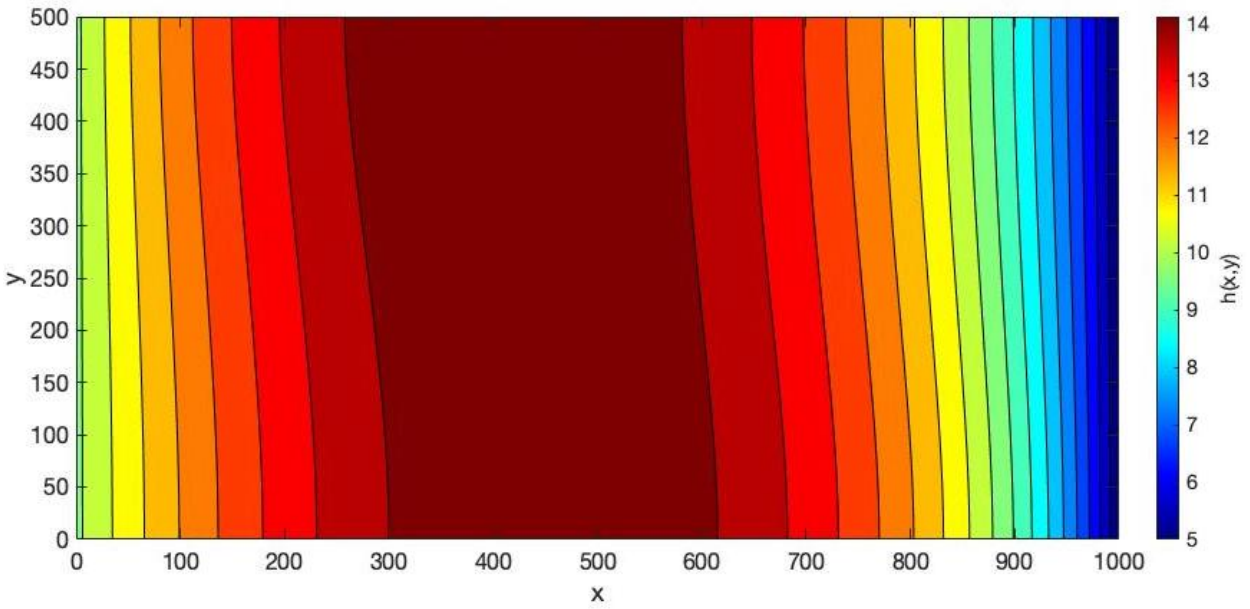

Fig. 5 Solution at 1000 days

Solution plots are shown above. We can see the left and right boundary conditions limiting the head values on the ends to 10m on the left end and 5m on the right end, for all number of days. The uniform recharge varies in time and eventually causes the head to climb greater than the original maximum, reaching a maximum of 14.68m towards the left middle of the domain after 1000 days. The bends in the contour lines showing a variation from bottom to top for the left to right gradient is due to the transmissivity function.

Fortran Code

```
! Andrew Navratil
! CE 791 Mini project – 2D Groundwater Flow

!*************************************************************************
! main solver program for 2D Groundwater Finite Difference
!*************************************************************************
program gw
implicit none
include "mpif.h"

    ! === constants ===
    real(8), parameter :: PI = 4.D0*DATAN(1.D0)

    real(8), parameter :: Lx = 1000., Ly = 500.     ! x,y grid lengths
    !real(8), parameter :: Lx = 10., Ly = 5.        ! x,y grid lengths

    ! 1D blocks will be groups of columns, so want the total num cols (sx)
    ! to be divisible by num procs (2,4,8,16 -> 16), keep it close to dx=1
    ! num rows (sy) can stay even, for dy=1
    ! we will print ghost nodes which are the boundaries for full grid
    real(8), parameter :: sx = 16*64, sy = 499.     ! total x,y nodes
    !real(8), parameter :: sx = 10, sy = 5.     ! total x,y nodes

    real(8), parameter :: dx = Lx/(sx+1), dy = Ly/(sy+1)    ! delta x, delta y

    !real(8), parameter :: tstart = 0., tstop = 10.
    !real(8), parameter :: tstart = 0., tstop = 100.
    real(8), parameter :: tstart = 0., tstop = 1000.

    ! experiment with different values (no cfl stability calculated)
    real(8), parameter :: dt = 0.001                ! time step

    ! K coeffs (see matlab code for calculation)
    real(8), parameter :: a = -0.001, b = -0.002, c = 0.000004, d = 2.0

    real(8), parameter :: Ss = 0.1              ! storage coefficient
    real(8), parameter :: Q0 = 0.001            ! recharge coeff
    real(8), parameter :: omega = PI/300.       ! used in recharge calc

    logical, parameter :: doprint = .false. ! debug and file prints

    ! === variables ===
    real(8), allocatable :: u(:,:),un(:,:)      ! sln(k),sln(k+1)
    real(8), allocatable :: K(:,:)              ! transmissivity

    integer nx,ny,xbase     ! num x,y nodes and start col per block/proc
    real(8) xval, yval      ! x,y coordinate values
    real(8) dudtx,dudty     ! du/dt x,y portions

    real(8) Q               ! areal specific recharge (changes with time)

```

```
52      real(8) t                       ! current time
53      integer i,j,tidx,jstart,jend    ! reserve i,j for indexing
54      integer numops                  ! count of ops for mflops
55
56      character(len=50) :: filename   ! filename for printing solution
57
58      ! === MPI variables ===
59      real(8) time1,time2,total_time,mflops,ierr
60      integer nprocs,iam,leftproc,rightproc,comm1d,mpistatus
61      integer coltype,rowtype,tag1,tag2
62
63      !============================================================
64
65      ! initialize MPI
66      call MPI_INIT(ierr)
67      call MPI_COMM_RANK(MPI_COMM_WORLD, iam, ierr)
68      call MPI_COMM_SIZE(MPI_COMM_WORLD, nprocs, ierr)
69      !nprocs = 16
70      !iam = 0
71
72      ! setup MPI topology
73      call MPI_CART_CREATE(MPI_COMM_WORLD, 1, nprocs, .false., &
74          .true., comm1d, ierr)
75      call MPI_COMM_RANK(comm1d, iam, ierr)
76      call MPI_CART_SHIFT(comm1d, 0, 1, leftproc, rightproc, ierr)
77
78      ! calculate block num cols per proc, num rows stays same for 1D
79      nx = sx/nprocs
80      ny = sy
81      xbase = iam*nx
82
83      ! allocate memory for solution and K
84      allocate (u(0:ny+1,0:nx+1),un(0:ny+1,0:nx+1))
85      allocate (K(0:ny+1,0:nx+1))
86
87      ! initialize message passing parameters
88      ! sln u changed to u(i,j) = u(y,x) so x-y columns correlate to
89      ! columns in the u variable too for contiguous memory access
90      call MPI_TYPE_CONTIGUOUS(ny+2,MPI_DOUBLE_PRECISION,coltype,ierr)
91      call MPI_TYPE_COMMIT(coltype,ierr)
92      ! for 2D domain decomposition (to do)
93      !call MPI_TYPE_VECTOR(nx+2,1,ny+2,MPI_DOUBLE_PRECISION,rowtype,ierr)
94      !call MPI_TYPE_COMMIT(rowtype,ierr)
95
96      tag1 = 1
97      tag2 = 2
98      if (iam.eq.0) then
99          leftproc = MPI_PROC_null
100     else
101         leftproc = iam-1
102     end if
103     if (iam.eq.nprocs-1) then
104         rightproc = MPI_PROC_NULL
105     else
```

```
106         rightproc = iam+1
107     end if
108 
109     ! init solution and K values (K doesn't change later) (per block)
110     do j = 0,nx+1
111     do i = 0,ny+1
112         xval = (xbase+j)*dx
113         yval = i*dy
114 
115         u(i,j) = 10.0-xval/200.0
116         !u(i,j) = 10.0-xval/2.0
117         K(i,j) = a*xval + b*yval + c*xval*yval + d
118 
119         !write(*,"(3I5,4E20.8)") iam,i,j,xval,yval,u(i,j),K(i,j)
120     end do
121     end do
122 
123     ! reset upper and lower rows using Neumann boundary condition
124     ! left,right boundary conditions already a product of the init sln eqn
125     do j=1,nx
126         u(ny+1,j) = u(ny,j)     ! upper
127         u(0,j) = u(1,j)         ! lower
128     end do
129 
130     ! start timer for mflops calculation
131     if (iam.eq.0) then
132         time1 = MPI_Wtime()
133     end if
134 
135     ! time marching - iterate solution from start to stop time
136     ! we already initialized at time tstart, so add dt
137     t = tstart + dt
138     tidx = 2
139     do while (t.le.tstop)
140         ! send first col to left neighbor
141         ! receive last col from right neighbor
142         call MPI_Sendrecv(u(0,1), 1, coltype, leftproc, tag1, &
143                 u(0,nx+1), 1, coltype, rightproc, tag1, &
144                 comm1d, mpistatus, ierr)
145         ! send last col to right neighbor
146         ! receive first col from left neighbor
147         call MPI_Sendrecv(u(0,nx), 1, coltype, rightproc, tag2, &
148                 u(0,0), 1, coltype, leftproc, tag2, &
149                 comm1d, mpistatus, ierr)
150 
151         ! calculate solution for next time step
152         Q = Q0*(1+sin(omega*t))
153         do j = 1,nx
154         do i = 1,ny
155             dudtx = (K(i,j+1)+K(i,j))*(u(i,j+1)+u(i,j))*(u(i,j+1)-u(i,j))
156             dudtx = dudtx - (K(i,j)+K(i,j-1))*(u(i,j)+u(i,j-1))*(u(i,j)-u(i,j-1))
157             dudtx = dudtx/(4*dx*dx)
158 
159             dudty = (K(i+1,j)+K(i,j))*(u(i+1,j)+u(i,j))*(u(i+1,j)-u(i,j))
```

```
160             dudty = dudty - (K(i,j)+K(i-1,j))*(u(i,j)+u(i-1,j))*(u(i,j)-u(i-1,j))
161             dudty = dudty/(4*dy*dy)
162 
163             un(i,j) = u(i,j) + (dt/Ss)*(dudtx+dudty+Q)
164         end do
165         end do
166 
167         ! now copy back into current solution
168         do j = 1,nx
169         do i = 1,ny
170             u(i,j) = un(i,j)
171         end do
172         end do
173 
174         ! insulated / no flow upper and lower boundaries
175         ! homogeneous Neumann BCs => du/dy = 0
176         ! this means ghost node is just same value as cell above/below
177         do j=1,nx
178             u(ny+1,j) = u(ny,j)     ! upper
179             u(0,j) = u(1,j)         ! lower
180         end do
181 
182         ! constant head left and right boundaries
183         ! nonhomogeneous Dirichlet BCs => u = val
184         if (iam.eq.0) then
185             do i=0,ny+1
186                 u(i,0) = 10.0           ! left
187             end do
188         elseif (iam.eq.nprocs-1) then
189             do i=0,ny+1
190                 u(i,nx+1) = 5.0         ! right
191             end do
192         end if
193 
194         ! print some sln values sometimes as we go along if doprint=true
195         !if (doprint) then
196             !if (mod(t,0.1).le.0.0001) then
197                 !write(*,"(I3,4E20.8)") iam,t,u(10,10),u(ny/2,nx/2),u(ny-10,nx-10)
198                 !write(*,"(I3,4E20.8)") iam,t,u(1,1),u(ny/2,nx/2),u(ny-1,nx-1)
199             !end if
200         !end if
201 
202         ! increment time
203         !t = t + dt
204         t = tstart + (tidx*dt)
205         tidx = tidx+1
206     end do
207 
208     ! end timer
209     if (iam.eq.0) then
210         time2 = MPI_Wtime()
211         total_time = time2-time1
212     end if
213 
```

```
214     ! print the final solution to file
215     if (doprint) then
216         write(filename,"(A17,I2.2,A1,I4.4,A4)") &
217             'minidata/slndata_',iam,'_',int(t),'.txt'
218         open(1, file = filename, status='replace')
219 
220         ! print the first ghost col at x=0 if proc 0
221         ! print the last ghost col at x=nx+1 if proc nprocs-1
222         ! always print the ghost rows at y=0,y=ny+1
223         jstart = 1
224         jend = nx
225         if (iam.eq.0) then
226             jstart = 0
227         elseif (iam.eq.nprocs-1) then
228             jend = nx+1
229         end if
230 
231         do j = jstart,jend
232         do i = 0,ny+1
233             !xval = (xbase+j)*dx
234             !yval = i*dy
235 
236             write(1,"(E20.8)") u(i,j)
237             !write(1,"(2I5,3E20.8)") i,j,xval,yval,u(i,j)
238             !write(*,"(3I5,3E20.8)") iam,i,j,xval,yval,u(i,j)
239         end do
240         end do
241 
242         close(1)
243     end if
244 
245     ! print mflops and execution time to gw.out file
246     if (iam.eq.0) then
247         numops = 14+14+5
248         mflops = dble(numops)*dble(nx*ny)*(tidx-2)*nprocs*1e-6/total_time
249 
250         write(*,*) 'nprocs =',nprocs
251         write(*,*) 'mflops =',mflops
252         write(*,*) 'total time =',total_time
253     end if
254 
255     ! finalize MPI
256     call MPI_FINALIZE(ierr)
257 
258 end program gw
```

Matlab Code

```
close all; clear all; clc;

%================================================================================
% plot mflops, exec time
if 1
    % fgw bsub executed on henry2
    % mflops, exec time, num procs printed to gw.out and compiled in gwout.txt
    % then manually entered in here for plotting
    procs = [1 2 4 8 16]
    mflops0010 = [6024 12913 26306 44328 68427]
    times0010 = [27.9877 13.0564 6.4093 3.8035 2.4639]
    mflops0100 = [6020 13264 29128 45476 76475]
    times0100 = [280.0738 127.1239 57.8888 37.0790 22.0489]
    mflops1000 = [6072 12646 28744 39367 55587]
    times1000 = [2776.5876 1333.3860 586.6255 428.3259 303.3453]

    semilogy(procs,times0010,'*--','MarkerSize',10,'DisplayName','t = 10d');
    hold on;
    semilogy(procs,times0100,'+--','MarkerSize',10,'DisplayName','t = 100d');
    semilogy(procs,times1000,'^--','MarkerSize',10,'DisplayName','t = 1000d');
    xlabel('Number of Processors');
    ylabel('Elapsed Time (s)');
    set(gca,'FontSize',16);
    legend('location','northeast');

    figure;
    plot(procs,mflops0010,'*--','MarkerSize',10,'DisplayName','t = 10d');
    hold on;
    plot(procs,mflops0100,'+--','MarkerSize',10,'DisplayName','t = 100d');
    plot(procs,mflops1000,'^--','MarkerSize',10,'DisplayName','t = 1000d');
    xlabel('Number of Processors');
    ylabel('Mflops');
    set(gca,'FontSize',16);
    legend('location','northwest');

end
%================================================================================
% print solution
if 0
    nprocs = 16
    u = []
    for i = 0:nprocs-1
        istr = sprintf('%02d',i)
        %d = importdata(['minidata0010/slndata_',istr,'_0010.txt']);
        %d = importdata(['minidata0100/slndata_',istr,'_0100.txt']);
        d = importdata(['minidata1000/slndata_',istr,'_1000.txt']);
        u = [u;d];
    end

    x = linspace(0,1000,16*64+2)';
    y = linspace(0,500,499+2)';
```

```
    z = reshape(u,[499+2,16*64+2]);

    colormap jet
    contourf(x,y,z,16);
    xlabel('x');
    ylabel('y');
    c = colorbar;
    c.Label.String = 'h(x,y)';
    set(gca,'FontSize',16);
    set(gcf,'Position',[500, 500, 1000, 800]);
    pbaspect([2 1 1]);

end
%=========================================================================
% calculate K coefficients
if 0
    % get values from sol.a, sol.b, sol.c, sol.d
    syms a b c d x y k
    f = a*x + b*y + c*x*y + d - k
    eqn1 = subs(f,{x,y,k},{0,0,2});
    eqn2 = subs(f,{x,y,k},{1000,0,1});
    eqn3 = subs(f,{x,y,k},{0,500,1});
    eqn4 = subs(f,{x,y,k},{1000,500,2});
    sol = solve(eqn1,eqn2,eqn3,eqn4);

    % confirm with matrix solution Ax=b
    A = [0 0 0 1; 1000 0 0 1; 0 500 0 1; 1000 500 1000*500 1];
    b = [2;1;1;2];
    x = A\b
end
```

gwout.txt

```
====================================
t=1000d

 nprocs =           1
 mflops =   6072.99075421041
 total time =   2776.58764481544

 nprocs =           2
 mflops =   12646.1437791611
 total time =   1333.38600206375

 nprocs =           4
 mflops =   28744.3862309495
 total time =   586.625540018082

 nprocs =           8
 mflops =   39367.6669417725
 total time =   428.325892925262

 nprocs =          16
 mflops =   55587.4416029699
 total time =   303.345335006714

====================================
t=100d

 nprocs =           1
 mflops =   6020.57052999881
 total time =   280.073777914047

 nprocs =           2
 mflops =   13264.2560499973
 total time =   127.123898029327

 nprocs =           4
 mflops =   29128.3513286355
 total time =   57.8887529373169

 nprocs =           8
 mflops =   45476.0421594135
 total time =   37.0789508819580

 nprocs =          16
 mflops =   76475.8055099775
 total time =   22.0488548278809

====================================
t=10d

 nprocs =           1
 mflops =   6024.26903013062
 total time =   27.9876639842987
```

```
nprocs =           2
mflops =   12913.6297318886
total time =   13.0563769340515

nprocs =           4
mflops =   26306.1339314909
total time =   6.40934991836548

nprocs =           8
mflops =   44328.9404592876
total time =   3.80350208282471

nprocs =          16
mflops =   68427.9390693282
total time =   2.46398210525513
```